



Hướng Dẫn Cài Đặt & Chạy Dự Án (Frontend Vue 3 - Vite - TypeScript)

Tài liệu này hướng dẫn chi tiết cấu trúc thư mục, các bước chuẩn bị môi trường, cài đặt thư viện, cấu hình và khởi chạy dự án Frontend của tập đoàn HDG (HDG Group Frontend) trên hệ điều hành Windows.

Cấu Trúc Thư Mục Dự Án

Dưới đây là sơ đồ cấu trúc thư mục của dự án và chức năng của từng thư mục/tệp tin chính:

```

HDG_Group_Frontend/
├─ .vscode/                # Cấu hình khuyên dùng cho VS Code (settings, extensions).
├─ public/                 # Các tài nguyên tĩnh công khai (không qua Vite build bundle).
│  └─ appsettings.json     # Cấu hình kết nối API Backend (được đọc động bởi Vite và client).
│  └─ favicon.ico          # Icon hiển thị trên tab trình duyệt.
│  └─ *.png/*.jpg          # Các file logo và hình ảnh thương hiệu HDG Group.
├─ src/                    # Thư mục chứa toàn bộ mã nguồn Vue 3.
│  └─ api/                 # Các dịch vụ gọi API Backend sử dụng Fetch API.
│     └─ apiConfig.ts      # Cấu hình Endpoint API động và tạo Header chung (AuthToken, ngrok bypass).
│     └─ auth.ts           # Xử lý đăng nhập, đăng ký, đăng xuất, lưu token vào LocalStorage.
│     └─ roscaService.ts   # Xử lý các API nghiệp vụ Hui (Rosca).
│     └─ tienNgaService.ts # Xử lý các API nghiệp vụ cho Tiến Nga.
│     └─ ...Service.ts     # Các API nghiệp vụ khác (Credit, Rental, Harvest, Vehicle...).
│  └─ assets/              # Chứa file CSS toàn cục, Tailwind CSS và tài nguyên tĩnh.
│     └─ main.css          # Khai báo Tailwind CSS và biến CSS tùy chỉnh.
│  └─ components/          # Chứa các component Vue theo từng mô-đun nghiệp vụ.
│     └─ auth/             # Giao diện đăng nhập, đăng ký, khôi phục mật khẩu.
│     └─ Authorization/    # Quản lý phân quyền tài khoản (chỉ dành cho Admin).
│     └─ Credit/           # Giao diện quản lý Hợp đồng vay, tín dụng.
│     └─ Ggomoosin/        # Giao diện quản lý nhân sự/sản xuất Ggomoosin.
│     └─ Harvest/          # Giao diện quản lý thu hoạch mủ cao su.
│     └─ Rental/           # Giao diện quản lý cho thuê xe/bất động sản.
│     └─ Rosca/            # Giao diện quản lý dây hui, đóng hui, hốt hui.
│     └─ TienNga/          # Giao diện quản lý tài chính, nhân sự, kho bãi Tiến Nga.
│     └─ icons/            # Các custom SVG Icon dạng component Vue.
│     └─ ...
│  └─ layouts/             # Chứa các component bố cục chung (Layout wrapper).
│     └─ Navigation.vue    # Thanh điều hướng Header & Menu điều khiển dự án.
│  └─ router/              # Cấu hình Router của ứng dụng (Vue Router).
│     └─ index.ts          # Định nghĩa các Route, Route Guards kiểm tra quyền và xác thực (Auth Guard).
│  └─ views/               # Các view/page chính của hệ thống.
│     └─ dashboard/        # Bố cục màn hình làm việc chính.
│        └─ Dashboard.vue  # Nhận diện route hiện tại và hiển thị mô-đun tương ứng.
│        └─ error/         # Các trang báo lỗi hệ thống.
│           └─ NotFound.vue # Trang lỗi 404 khi truy cập sai đường dẫn.
│  └─ App.vue              # Component gốc của ứng dụng (chứa <RouterView />).
│  └─ main.ts              # Điểm khởi chạy Vue App, khởi tạo Element Plus, router và styles.
│  └─ env.d.ts             # Khai báo kiểu TypeScript cho các biến môi trường.
├─ tsconfig.json           # File cấu hình chung của TypeScript.
├─ vite.config.ts          # File cấu hình build/dev server của Vite (đọc proxy từ appsettings.json).
├─ tailwind.config.js      # Cấu hình Tailwind CSS (colors, fonts, breakpoints...).
├─ postcss.config.js       # Cấu hình PostCSS để xử lý Tailwind CSS.
├─ wrangler.json           # Cấu hình deploy ứng dụng Single Page (SPA) lên Cloudflare Pages.
├─ package.json            # Khai báo các scripts chạy dự án và danh sách thư viện phụ thuộc.
└─ pnpm-lock.yaml          # File khóa phiên bản thư viện của pnpm.

```

1. Chuẩn Bị Môi Trường

Để đảm bảo dự án chạy ổn định và đồng nhất, hãy chuẩn bị trước các công cụ sau:

1. **Node.js**: Phiên bản yêu cầu:
 - **Node.js v20.19.0** hoặc **v22.12.0** trở lên.
 - Kiểm tra phiên bản hiện tại bằng lệnh: `node -v`
2. **PNPM**: Dự án sử dụng quản lý gói `pnpm` (hiệu năng cao và tiết kiệm dung lượng ổ đĩa).
 - Nếu chưa cài đặt, chạy lệnh: `npm install -g pnpm`
 - Kiểm tra phiên bản bằng lệnh: `pnpm -v`
3. **VS Code Extensions khuyến dùng**:
 - **Vue - Official (Volar)**: Hỗ trợ cú pháp Vue 3, SFC, TypeScript cho `.vue` (Hãy tắt Vetur nếu có).
 - **Tailwind CSS IntelliSense**: Gợi ý tên class Tailwind nhanh chóng.

2. Các Bước Cài Đặt Chi Tiết

Bước 2.1. Clone Code

Mở terminal (CMD, PowerShell hoặc Git Bash) và thực hiện clone dự án về máy:

```
git clone <URL_KHO_MA_NGUON_FRONTEND>
cd HDG_Group_Frontend
```

Bước 2.2. Cài Đặt Các Thư Viện (Dependencies)

Sử dụng `pnpm` để cài đặt toàn bộ gói thư viện cần thiết đã được định nghĩa trong `package.json`:

```
pnpm install
```

Lưu ý: Không dùng `npm install` hoặc `yarn install` để tránh tạo ra các file lock dư thừa và gây xung đột phiên bản.

3. Cấu Hình Endpoint API (`public/appsettings.json`)

Trong quá trình phát triển (Development), Vite Server được cấu hình làm Proxy chuyển hướng các yêu cầu `/api` sang server Backend thực tế để tránh lỗi chặn CORS.

Mở tệp `[public/appsettings.json](file:///d:/ExtraJob/HDG_Group_Frontend/public/appsettings.json)` và điều chỉnh thông tin kết nối tới Backend của bạn:

```
{
  "Backend": {
    "Addresss": "spore-unknown-crank.ngrok-free.dev", // Địa chỉ backend (localhost hoặc ngrok)
    "HttpPort": "", // Port HTTP (để trống nếu dùng ngrok/https mặc định)
    "HttpsPort": "443", // Port HTTPS (ví dụ: 443 cho ngrok, để trống nếu dùng HTTP thường)
    "APIPrefix": "/api/v1" // Tiền tố phiên bản API
  }
}
```

- **Trường hợp chạy Backend tại máy local (Localhost):**

```
"Backend": {
  "Addresss": "localhost",
  "HttpPort": "8000",
  "HttpsPort": "",
  "APIPrefix": "/api/v1"
}
```

(Lúc này Vite Proxy sẽ tự động chuyển hướng các yêu cầu `/api/...` về `http://localhost:8000/api/v1/...`)

4. Khởi Chạy Dự Án (Commands)

Tại thư mục gốc dự án, bạn có thể thực thi các lệnh sau thông qua `pnpm` :

4.1. Chạy Dự Án Ở Chế Độ Phát Triển (Development)

Chạy server phát triển cục bộ với tính năng tự động tải lại khi thay đổi code (Hot-Reload):

```
pnpm dev
```

Sau khi chạy thành công, terminal sẽ hiển thị đường link local (ví dụ: `http://localhost:5173/`). Bạn hãy click hoặc copy link này dán vào trình duyệt để sử dụng.

4.2. Biên Dịch và Tối Ưu Hóa Dự Án Cho Production (Build)

Để kiểm tra lỗi TypeScript và đóng gói dự án thành các tệp tĩnh tối ưu hóa (HTML/JS/CSS) trong thư mục `dist/` :

```
pnpm build
```

4.3. Xem Trước Bản Build Production (Preview)

Chạy thử giao diện tĩnh đã build ở môi trường local để kiểm tra trước khi deploy thực tế:

```
pnpm preview
```

4.4. Kiểm Tra Lỗi Kiểu Dữ Liệu (Type-Check)

Chỉ chạy trình kiểm tra kiểu dữ liệu TypeScript mà không build dự án:

```
pnpm type-check
```

5. Cơ Chế Hoạt Động Cốt Lõi của Dự Án

5.1. Cơ Chế Proxy và Bỏ Qua Cảnh Báo Ngrok

- **Vite Proxy**: Định nghĩa tại [vite.config.ts](file:///d:/ExtraJob/HDG_Group_Frontend/vite.config.ts), tự động chuyển hướng request từ `/api` về địa chỉ Backend thực tế cấu hình trong `appsettings.json`.
- **Header ngrok-skip-browser-warning**: Thiết lập tại [apiConfig.ts](file:///d:/ExtraJob/HDG_Group_Frontend/src/api/apiConfig.ts). Khi Backend sử dụng đường truyền ngrok miễn phí, header này bắt buộc phải có để ngrok không hiển thị trang cảnh báo trung gian (interstitial page) làm lỗi các lệnh gọi API từ JavaScript.

5.2. Điều Hướng và Bảo Mật Route (Vue Router)

Tập [router/index.ts](file:///d:/ExtraJob/HDG_Group_Frontend/src/router/index.ts) quản lý luồng truy cập bằng Route Guards `beforeEach`:

1. **Requires Auth** (`requiresAuth: true`): Nếu Route yêu cầu đăng nhập và người dùng chưa có token, router sẽ tự động chuyển hướng về trang `/login` và đính kèm tham số `redirect` để quay lại trang cũ sau khi đăng nhập thành công.
2. **Requires Admin** (`requiresAdmin: true`): Nếu Route yêu cầu quyền quản trị (như trang `/authorization`), hệ thống sẽ gọi API kiểm tra quyền Admin của user. Nếu không phải Admin, chuyển hướng ngược về màn hình tổng quan (`/tien-nga/overall`).

5.3. Bố Cục Trang Dashboard Linh Hoạt

Tập [views/dashboard/Dashboard.vue](file:///d:/ExtraJob/HDG_Group_Frontend/src/views/dashboard/Dashboard.vue) đóng vai trò điều phối chính:

- Dựa trên tiền tố URL (ví dụ: `/tien-nga/...` hoặc `/rosca/...`), `Dashbaord.vue` sẽ tự động chuyển đổi giữa các sub-dashboard tương ứng (`TienNgaIndex`, `RoscaIndex` ...).
- Từng mô-đun nghiệp vụ lại chứa một file `Index.vue` riêng (ví dụ: [TienNga/Index.vue](file:///d:/ExtraJob/HDG_Group_Frontend/src/components/TienNga/Index.vue)) để quản lý sidebar con, trạng thái màn hình desktop/mobile/tablet, và hiển thị linh hoạt các tab chức năng chi tiết qua Dynamic Components (`<component :is="activeView" />`).

6. Triển Khai Lên Cloudflare Pages (Deployment)

Dự án đã được cấu hình sẵn cho Cloudflare Pages thông qua tệp [wrangler.json](file:///d:/ExtraJob/HDG_Group_Frontend/wrangler.json).

Để deploy nhanh dự án lên Cloudflare bằng tài khoản của bạn, hãy chạy lệnh sau:

```
# Đăng nhập vào Cloudflare (chỉ cần làm một lần)
npx wrangler login

# Deploy thư mục dist lên Cloudflare Pages
npx wrangler pages deploy dist --project-name=hdg-group-2026
```

(Hoặc có thể kết nối kho lưu trữ GitHub của bạn trực tiếp với Cloudflare Pages để tự động hóa quy trình CI/CD khi đẩy code lên các nhánh main/production).